

Lab 6 - Usuarios, Procedimientos y Triggers

Misión 1:

```
-- 1. Rol vendedores
>Run
CREATE ROLE vendedores;

>Run
GRANT CONNECT ON DATABASE tienda_db TO vendedores;
>Run
GRANT USAGE ON SCHEMA public TO vendedores;
>Run
GRANT SELECT ON TABLE productos TO vendedores;
>Run
GRANT INSERT ON TABLE pedidos, detalle_pedido TO vendedores;

>Run
CREATE ROLE maria LOGIN PASSWORD 'vendedora1';
>Run
CREATE ROLE juan LOGIN PASSWORD 'vendedor2';

>Run
GRANT vendedores TO maria;
>Run
GRANT vendedores TO juan;

tienda_db=# CREATE ROLE vendedores;

GRANT CONNECT ON DATABASE tienda_db TO vendedores;
GRANT USAGE ON SCHEMA public TO vendedores;
GRANT SELECT ON TABLE productos TO vendedores;
GRANT INSERT ON TABLE pedidos, detalle_pedido TO vendedores;

CREATE ROLE maria LOGIN PASSWORD 'vendedora1';
CREATE ROLE juan LOGIN PASSWORD 'vendedor2';

GRANT vendedores TO maria;
GRANT vendedores TO juan;

CREATE ROLE
GRANT
GRANT
GRANT
GRANT
CREATE ROLE
CREATE ROLE
GRANT ROLE
GRANT ROLE
```

Explicación: en esta misión, se crea un rol de vendedores y los usuarios de maría y juan, luego se hacen permisos para los roles como conectarse, ver productos e insertar nuevas cosas en la tabla pedidos o detalles de pedido, finalmente se le asignan los roles de vendedores a los usuarios de maría y juan. Es importante debido a que se le da a Juan y Maria la capacidad y los permisos de vendedores. Al hacer un role de vendedores se evita tener que signar manualmente los mismos permisos a diferentes usuarios, por lo que facilita el trabajo al querer darle los mismos permisos a un usuario que pueda crearse en el futuro.

Misión 2:

```
-- 2. Auditor
>Run
CREATE ROLE auditor LOGIN PASSWORD 'auditorExterno'
VALID UNTIL '2026-11-10';

>Run
GRANT CONNECT ON DATABASE tienda_db TO auditor;
>Run
GRANT USAGE ON SCHEMA public TO auditor;
>Run
GRANT SELECT ON ALL TABLES IN SCHEMA public TO auditor;

tienda_db=# CREATE ROLE auditor LOGIN PASSWORD 'auditorExterno'
VALID UNTIL '2026-11-10';

GRANT CONNECT ON DATABASE tienda_db TO auditor;
GRANT USAGE ON SCHEMA public TO auditor;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO auditor;

CREATE ROLE
GRANT
GRANT
GRANT
```

Explicación: para este caso, se necesitaba crear un auditor que tenga acceso por un tiempo determinado, por lo que se crea el rol de auditor y se le asigna un tiempo límite para acceder, después se le dan permisos para que pueda acceder a las tablas solo para verlas. Al hacer esto se consigue que el auditor pueda acceder a la información solo por el tiempo especificado y que lo único que pueda hacer es ver la información, evitando que edite algún dato.

Misión 3:

```
-- 3. Revocar permisos en vendedores para acceder a clientes
>Run
GRANT SELECT ON TABLE clientes TO vendedores;

>Run
REVOKE SELECT ON TABLE clientes FROM vendedores;
```

```
tienda_db=# GRANT SELECT ON TABLE clientes TO vendedores;

REVOKE SELECT ON TABLE clientes FROM vendedores;
GRANT
REVOKE
```

Explicación: en esta misión se quitan permisos a vendedores, específicamente el permiso de ver la tabla. Al hacer esto se consigue que los vendedores dejen de ver la información de contacto de los clientes.

Misión 4:

```
-- 4. Vista clientes activos para vendedores
>Run | Select
CREATE VIEW vista_clientesActivos AS
SELECT id_cliente, nombre, email FROM clientes
WHERE activo = TRUE;

>Run
GRANT SELECT ON vista_clientesActivos TO vendedores;
```

```
tienda_db=# CREATE VIEW vista_clientesActivos AS
SELECT id_cliente, nombre, email FROM clientes
WHERE activo = TRUE;

GRANT SELECT ON vista_clientesActivos TO vendedores;
CREATE VIEW
GRANT
```

Explicación: en este caso se crea una vista con los clientes activos para los vendedores y se le da permiso de ver. Al hacer esto, los vendedores pueden ver únicamente los clientes que estén activos en el momento y se evita que puedan ver la tabla directamente.

Misión 5:

```
-- 5. Revocar permisos en vendedores para insertar en la tabla
>Run
REVOKE INSERT ON TABLE pedidos FROM vendedores;
```

```
tienda_db=# REVOKE INSERT ON TABLE pedidos FROM vendedores;

REVOKE
```

Explicación: en esta misión se le quita a los vendedores el permiso de insertar datos, evitando que puedan añadir algo sin validarlo primero.

Misión 6:

```
-- 6. Función que devuelve productos
>Run | Select
CREATE OR REPLACE FUNCTION obtener_productos_disponibles()
RETURNS TABLE(id_producto INT, nombre VARCHAR, precio DECIMAL, stock INT)
AS $$ BEGIN RETURN QUERY SELECT p.id_producto, p.nombre, p.precio, p.stock
FROM productos p WHERE p.stock > 0;

END;
$$ LANGUAGE plpgsql;

>Run | +Tab | JSON
SELECT * FROM obtener_productos_disponibles();
```

```
tienda_db=# CREATE OR REPLACE FUNCTION obtener_productos_disponibles()
RETURNS TABLE(id_producto INT, nombre VARCHAR, precio DECIMAL, stock INT)
AS $$ BEGIN RETURN QUERY SELECT p.id_producto, p.nombre, p.precio, p.stock
FROM productos p WHERE p.stock > 0;

END;
$$ LANGUAGE plpgsql;

SELECT * FROM obtener_productos_disponibles();
CREATE FUNCTION
id_producto | nombre | precio | stock
-----+-----+-----+-----
1 | Laptop | 1200.00 | 10
2 | Mouse | 25.00 | 50
3 | Teclado | 45.00 | 30
4 | Monitor | 300.00 | 15
5 | Auriculares | 80.00 | 25
6 | Webcam | 60.00 | 20
7 | Silla Gamer | 250.00 | 8
8 | Escritorio | 400.00 | 5
9 | USB 64GB | 15.00 | 100
10 | Disco SSD 1TB | 150.00 | 12
(10 rows)
```

Explicación: se crea una función para devolver los productos que tengan aunque sea una unidad en su stock. Al hacer esto se evita ver inventario que ya esté descontinuado o no esté disponible.

Misión 7:

```
-- 7. Crear función para activar cliente
▷Run | □Select
CREATE OR REPLACE FUNCTION activar_cliente(p_id_cliente INT)
RETURNS TEXT AS $$ DECLARE estado_actual BOOLEAN;

BEGIN
SELECT activo INTO estado_actual
FROM clientes WHERE id_cliente = p_id_cliente;

IF estado_actual IS NULL THEN
    RETURN 'Cliente no existe';
END IF;

IF estado_actual = TRUE THEN
    RETURN 'Cliente ya está activo';
END IF;

UPDATE clientes SET activo = TRUE
WHERE id_cliente = p_id_cliente;

RETURN 'cliente activado correctamente';

END;
$$ LANGUAGE plpgsql;

▷Run | +Tab | JSON
SELECT activar_cliente(1);
```

```
tienda_db=# CREATE OR REPLACE FUNCTION activar_cliente(p_id_cliente INT)
RETURNS TEXT AS $$ DECLARE estado_actual BOOLEAN;

BEGIN
SELECT activo INTO estado_actual
FROM clientes WHERE id_cliente = p_id_cliente;

IF estado_actual IS NULL THEN
    RETURN 'Cliente no existe';
END IF;

IF estado_actual = TRUE THEN
    RETURN 'Cliente ya está activo';
END IF;

UPDATE clientes SET activo = TRUE
WHERE id_cliente = p_id_cliente;

RETURN 'Cliente activado correctamente';

END;
$$ LANGUAGE plpgsql;

SELECT activar_cliente(1);
CREATE FUNCTION
    activar_cliente
-----
    Cliente ya está activo
(1 row)
```

Explicación: se crea una función que recibe un identificador de cliente y lo activa si el cliente existe y está activo, evitando que los clientes tengan que activarse manualmente y que pueda haber errores al no validar si el cliente no existe o no está activo.

Misión 8:

```
-- 8. Verificar disponibilidad del producto y estado de cliente
▷Run | □Select
CREATE OR REPLACE PROCEDURE crear_pedido_seguro(p_id_cliente INT, p_id_producto INT, p_cantidad INT)
LANGUAGE plpgsql AS
$$
DECLARE
    v_stock INT;
    v_precio DECIMAL;
    v_activo BOOLEAN;
    v_total DECIMAL;
    v_id_pedido INT;
BEGIN
    SELECT activo INTO v_activo
    FROM clientes WHERE id_cliente = p_id_cliente;

    IF v_activo IS DISTINCT FROM TRUE THEN
        RAISE EXCEPTION
        'Cliente inactivo o existente';
    END IF;

    SELECT stock, precio INTO v_stock, v_precio
    FROM productos WHERE id_producto = p_id_producto;

    IF v_stock < p_cantidad THEN
        RAISE EXCEPTION
        'Stock insuficiente';
    END IF;

    v_total := v_precio * p_cantidad;

    INSERT INTO pedidos(id_cliente, total) VALUES(p_id_cliente, v_total)
    RETURNING id_pedido INTO v_id_pedido;

    INSERT INTO detalle_pedido(id_pedido, id_producto, cantidad, subtotal) VALUES(v_id_pedido, p_id_producto, p_cantidad, v_total);
END;
$$;
```

```
tienda_db=# CREATE OR REPLACE PROCEDURE crear_pedido_seguro(p_id_cliente INT, p_id_producto INT, p_cantidad INT)
LANGUAGE plpgsql AS
$$
DECLARE
    v_stock INT;
    v_precio DECIMAL;
    v_activo BOOLEAN;
    v_total DECIMAL;
    v_id_pedido INT;
BEGIN
    SELECT activo INTO v_activo
    FROM clientes WHERE id_cliente = p_id_cliente;

    IF v_activo IS DISTINCT FROM TRUE THEN
        RAISE EXCEPTION
        'Cliente inactivo o existente';
    END IF;

    SELECT stock, precio INTO v_stock, v_precio
    FROM productos WHERE id_producto = p_id_producto;

    IF v_stock < p_cantidad THEN
        RAISE EXCEPTION
        'Stock insuficiente';
    END IF;

    v_total := v_precio * p_cantidad;

    INSERT INTO pedidos(id_cliente, total) VALUES(p_id_cliente, v_total)
    RETURNING id_pedido INTO v_id_pedido;

    INSERT INTO detalle_pedido(id_pedido, id_producto, cantidad, subtotal) VALUES(v_id_pedido, p_id_producto, p_cantidad, v_total);
END;
$$;
CREATE PROCEDURE
```

Explicación: en esta misión se crea una función que verifica la disponibilidad del producto y el estado del cliente al querer insertar un pedido, evitando que se inserten pedidos con productos no disponibles o a clientes que estén inactivos.

Misión 9:

```
-- 9. Hacer segura la creación de pedidos
▷Run
GRANT EXECUTE ON PROCEDURE crear_pedido_seguro TO vendedores;

▷Run
REVOKE INSERT ON detalle_pedido FROM vendedores;
GRANT
REVOKE
```

Explicación: en este caso se crea un permiso para crear un pedido seguro y se le asigna a vendedores, luego se quita el permiso de insertar nueva información en detalle pedido por parte de los vendedores, de esta manera se hace más segura la creación de pedidos.

Misión 10:

```
-- 10. Validación de inventario
▷Run | Select
CREATE OR REPLACE FUNCTION validar_stock()
RETURNS TRIGGER AS
$$
DECLARE
    v_stock INT;
BEGIN
    SELECT stock INTO v_stock
    FROM productos WHERE id_producto = NEW.id_producto;

    IF v_stock <= 0 THEN
        RAISE EXCEPTION
        'Producto sin inventario';
    END IF;

    RETURN NEW;
END;
$$
LANGUAGE plpgsql;

▷Run | Select
CREATE TRIGGER trg_validar_stock
BEFORE INSERT ON detalle_pedido
FOR EACH ROW EXECUTE FUNCTION validar_stock();

tienda_db=# CREATE OR REPLACE FUNCTION validar_stock()
RETURNS TRIGGER AS
$$
DECLARE
    v_stock INT;
BEGIN
    SELECT stock INTO v_stock
    FROM productos WHERE id_producto = NEW.id_producto;

    IF v_stock <= 0 THEN
        RAISE EXCEPTION
        'Producto sin inventario';
    END IF;

    RETURN NEW;
END;
$$
LANGUAGE plpgsql;

CREATE TRIGGER trg_validar_stock
BEFORE INSERT ON detalle_pedido
FOR EACH ROW EXECUTE FUNCTION validar_stock();
CREATE FUNCTION
```

Explicación: en este caso se crea un trigger que ejecuta una función antes de insertar en detalle de pedido y que se encarga de validar el stock, evitando que se inserte un producto sin inventario.

Misión 11:

```
-- 11. Descontar inventario
▷Run | □Select
CREATE OR REPLACE FUNCTION descontar_stock()
RETURNS TRIGGER AS
$$

BEGIN
    UPDATE productos
    SET stock = stock - NEW.cantidad
    WHERE id_producto = NEW.id_producto;

    RETURN NEW;
END;
$$
LANGUAGE plpgsql;

▷Run | □Select
CREATE TRIGGER trg_desconectar_stock
AFTER INSERT ON detalle_pedido
FOR EACH ROW EXECUTE FUNCTION descontar_stock();
```

```
tienda_db=# CREATE OR REPLACE FUNCTION descontar_stock()
RETURNS TRIGGER AS
$$

BEGIN
    UPDATE productos
    SET stock = stock - NEW.cantidad
    WHERE id_producto = NEW.id_producto;

    RETURN NEW;
END;
$$
LANGUAGE plpgsql;

CREATE TRIGGER trg_desconectar_stock
AFTER INSERT ON detalle_pedido
FOR EACH ROW EXECUTE FUNCTION descontar_stock();
CREATE FUNCTION
CREATE TRIGGER
```

Explicación: en esta misión se crea un trigger que se ejecuta al querer insertar información en detalle pedido, después de hacerlo y que se encarga de actualizar el stock, evitando que haya inconsistencia entre los productos y la información guardada en el sistema.

Misión 12:

```
-- 12. Actualizar total del pedido
▷Run | □Select
CREATE OR REPLACE FUNCTION actualizar_total_pedido()
RETURNS TRIGGER AS
$$

BEGIN
    UPDATE pedidos
    SET total = (
        SELECT COALESCE(SUM(subtotal), 0)
        FROM detalle_pedido WHERE id_pedido = NEW.id_pedido
    )
    WHERE id_pedido = NEW.id_pedido;

    RETURN NEW;
END;
$$
LANGUAGE plpgsql;

▷Run | □Select
CREATE TRIGGER trg_actualizar_total
AFTER INSERT ON detalle_pedido
FOR EACH ROW EXECUTE FUNCTION actualizar_total_pedido();
```

```
tienda_db=# CREATE OR REPLACE FUNCTION actualizar_total_pedido()
RETURNS TRIGGER AS
$$

BEGIN
    UPDATE pedidos
    SET total = (
        SELECT COALESCE(SUM(subtotal), 0)
        FROM detalle_pedido WHERE id_pedido = NEW.id_pedido
    )
    WHERE id_pedido = NEW.id_pedido;

    RETURN NEW;
END;
$$
LANGUAGE plpgsql;

CREATE TRIGGER trg_actualizar_total
AFTER INSERT ON detalle_pedido
FOR EACH ROW EXECUTE FUNCTION actualizar_total_pedido();
CREATE FUNCTION
CREATE TRIGGER
```

Explicación: se crea una función para actualizar el total del pedido, la cual es lanzada por un trigger luego de insertar nueva información en detalle pedido y la cual se encarga de actualizar pedidos, evitando que haya inconsistencias de información.

Misión 13:

```
-- 13. Auditoria de inventario
>Run| Select
CREATE OR REPLACE FUNCTION auditoria_inventario()
RETURNS TRIGGER AS
$$
BEGIN
    INSERT INTO auditoria_stock(id_producto, stock_anterior, stock_nuevo, fecha)
    VALUES(OLD.id_producto, OLD.stock, NEW.stock, NOW());

    RETURN NEW;
END;
$$
LANGUAGE plpgsql;

>Run| Select
CREATE TRIGGER trg_auditoria_inventario
AFTER UPDATE ON productos
FOR EACH ROW EXECUTE FUNCTION auditoria_inventario();
CREATE FUNCTION
CREATE TRIGGER
```

Explicación: en este caso se hace un trigger que se ejecuta luego de actualizar un producto y que se encarga de registrar cada cambio realizado en las unidades de un producto, evitando que haya cambios sin ser registrados.

Misión 14:

```
-- 14. Validación de monto de pago
>Run| Select
CREATE OR REPLACE FUNCTION validar_pago()
RETURNS TRIGGER AS
$$
DECLARE
    v_total DECIMAL;
BEGIN
    SELECT total INTO v_total
    FROM pedidos WHERE id_pedido = NEW.id_pedido;

    IF NEW.monto <> v_total THEN
        RAISE EXCEPTION
        'El monto no coincide con el total del pedido';
    END IF;

    RETURN NEW;
END;
$$
LANGUAGE plpgsql;

>Run| Select
CREATE TRIGGER trg_validar_pago
BEFORE INSERT ON pagos
FOR EACH ROW EXECUTE FUNCTION validar_pago();
CREATE FUNCTION
CREATE TRIGGER
```

Explicación: en esta misión se hace un trigger que se lanza antes de insertar un pago y que se encarga de validar que el monto de un pago coincida con el total del pedido, evitando que haya inconsistencias entre los pagos registrados y los totales en los pedidos.